

MEDTRACK: AWS Cloud-Enabled Healthcare Management System

Project Description:

In today's fast-evolving healthcare landscape, efficient communication and coordination between doctors and patients are crucial. Med Track is a cloud-based healthcare management system that streamlines patient doctor interactions by providing a centralized platform for booking appointments, managing medical histories, and enabling diagnosis submissions. To address these challenges, the project utilizes Flask for backend development, AWS EC2 for hosting, and DynamoDB for managing data. Med-track allows patients to register, log in, book appointments, and submit diagnosis reports online. The system ensures real-time notifications, enhancing communication between doctors and patients regarding appointments and medical submissions. Additionally, AWS Identity and Access Management (IAM) is employed to ensure secure access control to AWS resources, allowing only authorized users to access sensitive data. This cloud-based solution improves accessibility and efficiency in healthcare services for all users..

Scenario 1: Efficient Appointment Booking System for Patients

In the MEDTRACK system, AWS EC2 provides a reliable infrastructure to manage multiple patients accessing the platform simultaneously. For example, a patient can log in, navigate to the appointment booking page, and easily submit a request for an appointment. Flask handles backend operations, efficiently retrieving and processing user data in real-time. The cloud-based architecture allows the platform to handle a high volume of appointment requests during peak periods, ensuring smooth operation without delays.

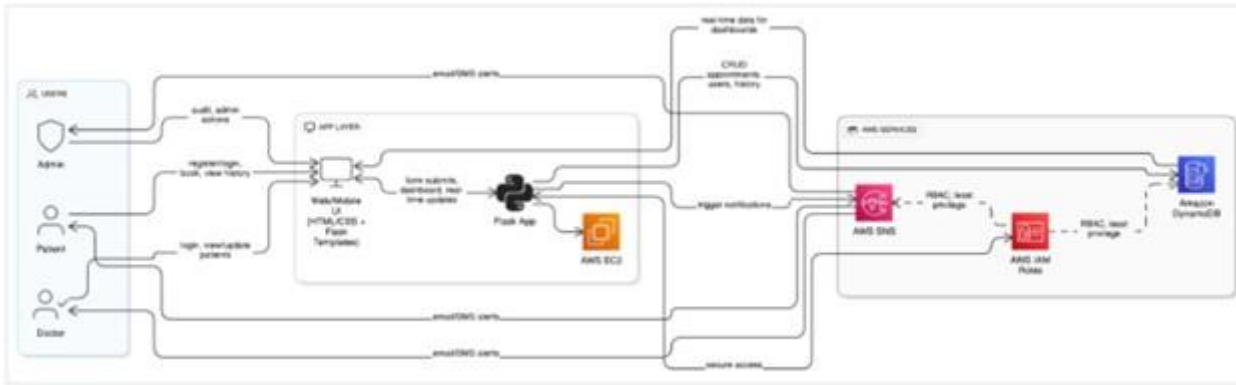
Scenario 2: Secure User Management with IAM

The MEDTRACK system provides doctors and patients with easy access to medical histories and relevant resources. For example, a doctor logs in to view a patient's medical history and upcoming appointments. They can quickly access, and update records as needed. Flask manages real-time data fetching from DynamoDB, while EC2 hosting ensures the platform performs seamlessly even when multiple users access it simultaneously, offering a smooth and uninterrupted user experience.

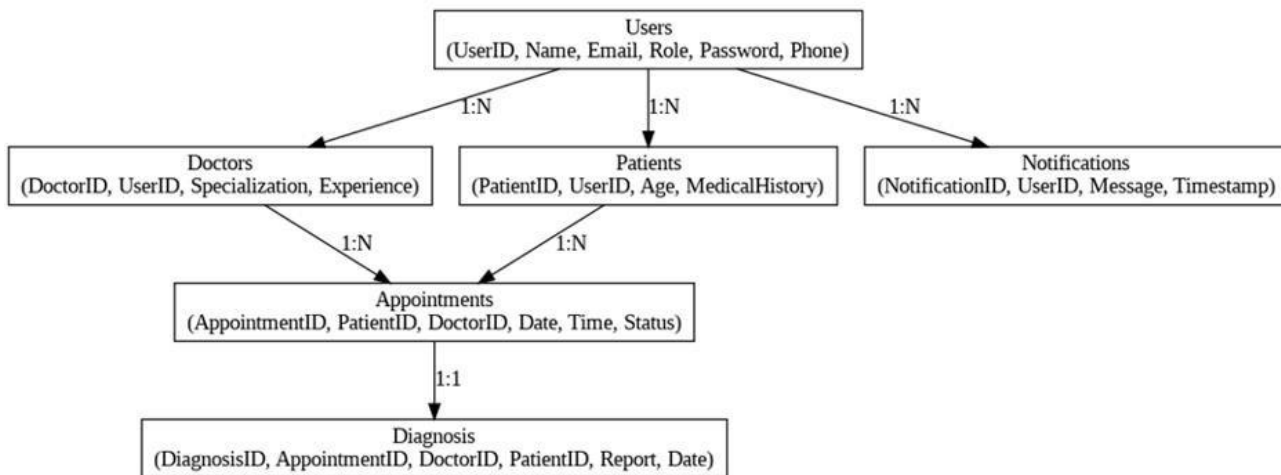
Scenario 3: Easy Access to Medical History and Resources

MEDTRACK features a dynamic user dashboard that displays all past and upcoming bookings for the logged-in user. For example, a user who has booked a flight and a hotel can view these bookings categorized by type, along with dates, price, and cancellation options. Flask fetches this data from AWS DynamoDB, which persistently stores all user bookings. The dashboard UI, powered by responsive HTML/CSS and Flask templates, ensures users can review or manage bookings anytime, from any device, with real-time updates and quick cancellation workflows.

AWS ARCHITECTURE



Entity Relationship (ER)Diagram:



Pre-requisites:

1. **AWS Account Setup:** [AWS Account Setup](#)
2. **Understanding IAM:** [IAM Overview](#)
3. **Amazon EC2 Basics:** [EC2 Tutorial](#)
4. **DynamoDB Basics:** [DynamoDB Introduction](#)
5. **SNS Overview:** [SNS Documentation](#)
6. **Git Version Control:** [Git Documentation](#)

Project WorkFlow:

1. AWS Account Setup and Login

Activity 1.1: Set up an AWS account if not already done.

Activity 1.2: Log in to the AWS Management Console

2. DynamoDB Database Creation and Setup

Activity 2.1: Create a DynamoDB Table.

Activity 2.2: Configure Attributes for travel-Users and Bookings Tables.

3. SNS Notification Setup

Activity 3.1: Create an SNS Topic named BookingConfirmation for sending notifications.

Activity 3.2: Subscribe email to the topic to receive booking and cancellation alerts.

4. Backend Development and Application Setup

Activity 4.1: Develop the Backend Using Flask.

Activity 4.2: Integrate AWS Services Using boto3.

5. IAM Role Setup

Activity 5.1: Create IAM Role

Activity 5.2: Attach Policies

6. EC2 Instance Setup

Activity 6.1: Launch an EC2 instance to host the Flask application.

Activity 6.2: Configure security groups for HTTP, and SSH access.

7. Deployment on EC2

Activity 7.1: Upload Flask

Files **Activity 7.2:** Run the

Flask App

8. Testing and Deployment

Activity 8.1: Conduct functional testing to verify user registration, login, book requests, and notifications.

Epic 1 : Web Application Development And Setup

● Activity 1.1: Develop Backend Using Flask.

The backend of the MEDTRACK system is developed using the Flask framework in Python. Flask is used to handle application routing, manage user requests, process appointment booking operations, and interact with AWS services such as DynamoDB and SNS. The backend also manages patient records, doctor information, and diagnosis submissions.

● Activity 1.2: Create Frontend Pages.

the frontend of the application is developed using HTML, CSS, and JavaScript to provide an interactive user interface. The system includes pages such as home page, login page, registration page, patient dashboard, doctor dashboard, appointment booking page, and diagnosis submission page. These pages allow users to interact with the system easily.

● Activity 1.3: Implement User Authentication

User authentication is implemented to allow patients and doctors to securely access the system. Users can register with their details and log in using their credentials. Passwords are securely stored and validated during login to ensure safe access to the healthcare management system.

Activity 1.4 – Configure Application Structure

The project follows a structured directory format. A templates folder is used to store HTML pages, while the static folder contains CSS and JavaScript files. The main backend logic is implemented in the app.py file. This structure ensures better organization and maintainability of the project.

Activity 1.5 – Test Web Application Locally

Before deploying the application to the cloud, the web application is tested locally. The Flask server is executed on the local machine to verify the functionality of user registration, login, appointment booking, and diagnosis submission. This ensures that the system works correctly before deployment.

Epic 2 : AWS Account Setup

● Activity 1.1: Set up an AWS account if not already done.

- Sign up for an AWS account and configure billing settings



Complete your AWS registration

Millions of customers are using AWS cloud solutions to build applications with increased flexibility, scalability, security, and reliability

[Complete sign-up](#)

AWS Free Tier

Use Amazon EC2, S3, and more— free for a full year

Customer success stories

Learn how companies are using AWS to solve their biggest challenges

Contact us

Reach out to us for any AWS related questions



Explore Free Tier products with a new AWS account.

To learn more, visit aws.amazon.com/free.



Sign up for AWS

Root user email address

Used for account recovery and as described in the [AWS Privacy Notice](#)

AWS account name

Choose a name for your account. You can change this name in your account settings after you sign up.

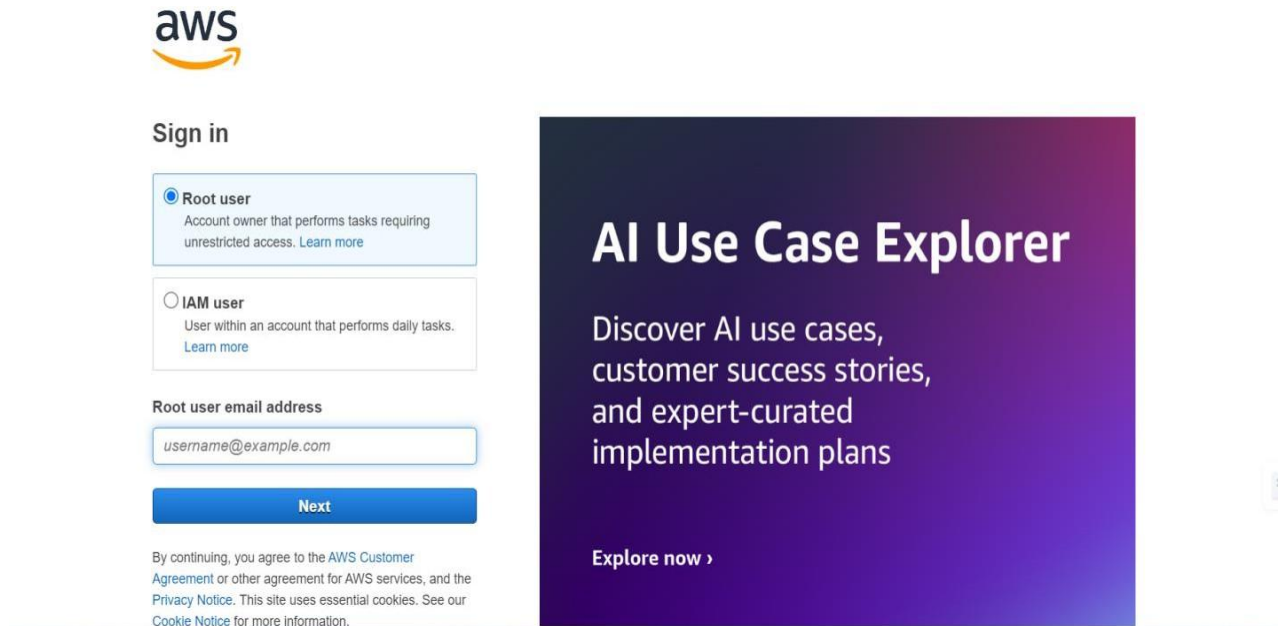
[Verify email address](#)

OR

[Sign in to an existing AWS account](#)

- **Activity 1.2: Log in to the AWS Management Console**

- After setting up your account, log in to the [AWS Management Console](#).

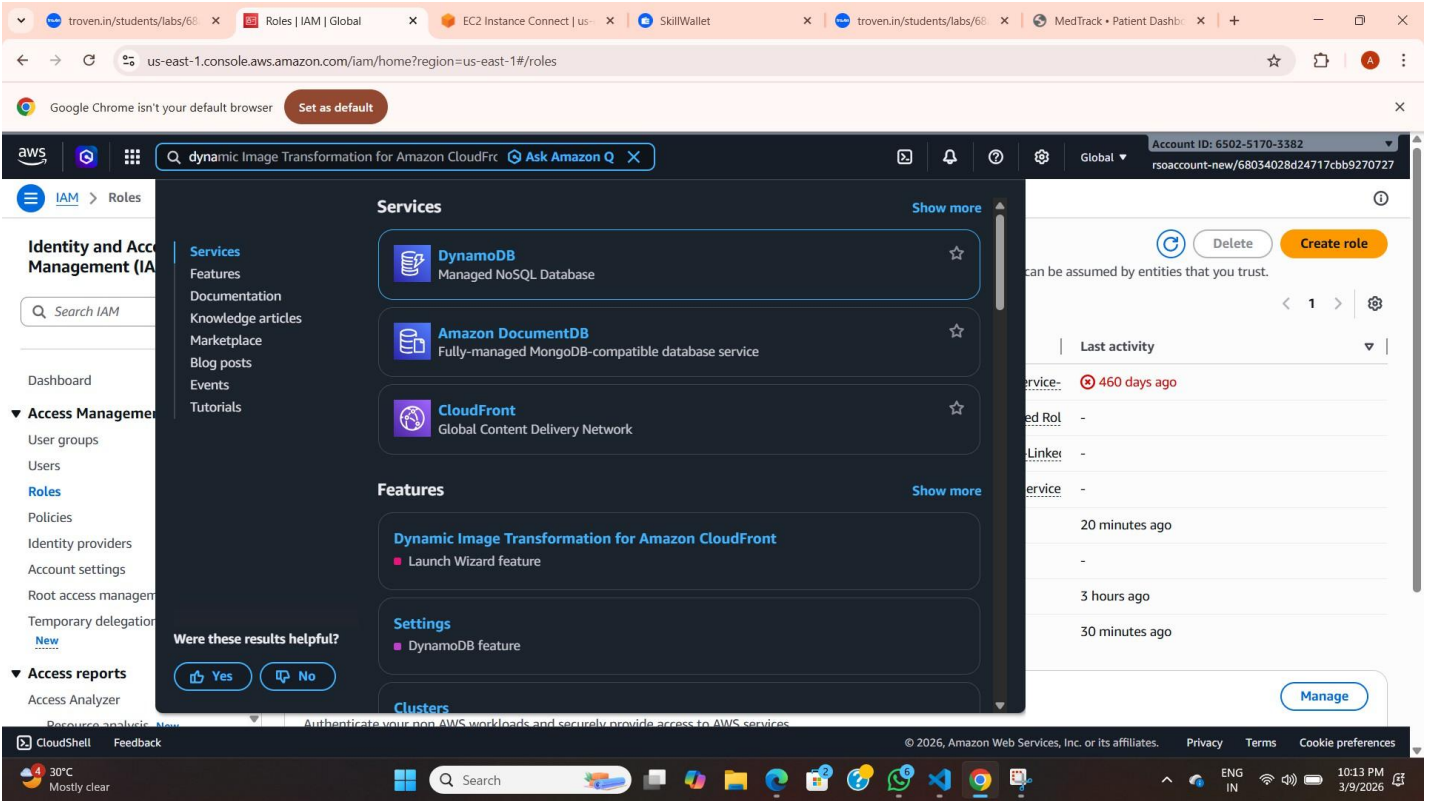


The image shows the AWS sign-in page. At the top left is the AWS logo. Below it, the text "Sign in" is displayed. There are two radio button options: "Root user" (selected) and "IAM user". The "Root user" option has a description: "Account owner that performs tasks requiring unrestricted access. [Learn more](#)". The "IAM user" option has a description: "User within an account that performs daily tasks. [Learn more](#)". Below these options is a text input field labeled "Root user email address" containing the placeholder text "username@example.com". A blue "Next" button is positioned below the input field. At the bottom left, there is a small disclaimer: "By continuing, you agree to the [AWS Customer Agreement](#) or other agreement for AWS services, and the [Privacy Notice](#). This site uses essential cookies. See our [Cookie Notice](#) for more information." On the right side of the page, there is a large purple and blue gradient box with the text "AI Use Case Explorer" in white, followed by "Discover AI use cases, customer success stories, and expert-curated implementation plans" and a link "Explore now >".

Epic 3: DynamoDB Database Creation and Setup

Activity 3.1: Navigate to the DynamoDB

To store user and appointment data for the MEDTRACK system, Amazon DynamoDB is used as the database service. In the AWS Management Console, the DynamoDB service is searched and selected to begin database creation and management.



The screenshot shows the AWS IAM console with the search results for 'dynamic Image Transformation for Amazon CloudFront'. The results are categorized into Services, Features, and Settings. The 'Dynamic Image Transformation for Amazon CloudFront' feature is highlighted. The left sidebar shows the 'IAM' menu with 'Roles' selected. The right sidebar shows the 'Create role' button and a table of roles.

Role	Last activity
service-	460 days ago
ed Rol	-
Linker	-
service	-
	20 minutes ago
	-
	3 hours ago
	30 minutes ago

Activity 3.2 – Create DynamoDB Tables

Two DynamoDB tables are created to store the system data. The Users table stores user information such as patient and doctor details. The Appointments table stores appointment booking details including appointment ID, doctor ID, patient ID, date, and status. These tables allow the system to efficiently manage healthcare records.

troven.in/students/labs/68 x List tables | Amazon Dynam x EC2 Instance Connect | us x SkillWallet x troven.in/students/labs/68 x MedTrack • Patient Dashb x +

us-east-1.console.aws.amazon.com/dynamodbv2/home?region=us-east-1#tables

Google Chrome isn't your default browser [Set as default](#)

aws Search [Alt+S]

United States (N. Virginia) Account ID: 6502-5170-3382 rsoaccount-new/68034028d24717cbb9270727

DynamoDB > Tables

Share your feedback on Amazon DynamoDB
Your feedback is an important part of helping us provide a better customer experience. Take this short survey to let us know how we're doing. [Share feedback](#)

Tables (4) Info Last updated March 9, 2026, 22:14 (UTC+5:30) [Actions](#) [Delete](#) [Create table](#)

Find tables Filter by tag Any tag key Filter by tag value Any tag value

☐	Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Favorite	Read capacity
☐	Appointments	Active	id (S)	-	0	0	Off	☆	On-demand
☐	Medications	Active	id (S)	-	0	0	Off	☆	On-demand
☐	Prescriptions	Active	id (S)	-	0	0	Off	☆	On-demand
☐	Users	Active	email (S)	-	0	0	Off	☆	On-demand

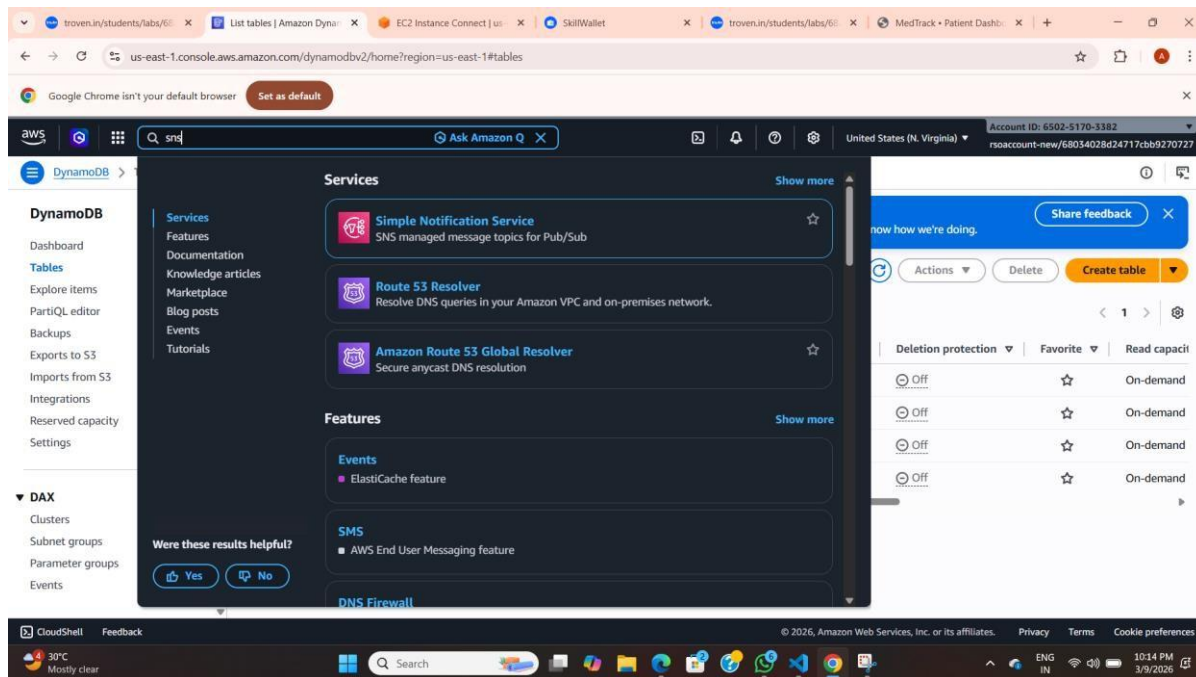
CloudShell Feedback © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

30°C Mostly clear Search 10:14 PM 3/9/2026

Epic 4: SNS Notification Setup

Activity 4.1 – Create SNS Topic for Sending Email Notifications

Amazon Simple Notification Service (SNS) is used in the MedTrack system to send notifications to users. An SNS topic is created in the AWS console to manage email notifications. This topic allows the system to send automated notifications when important actions occur such as appointment booking or diagnosis updates.



Activity 4.2 – Subscribe Users and Admin to SNS Topic

After creating the SNS topic, email addresses of users and administrators are subscribed to the topic. Once subscribed, they will receive notifications whenever the system publishes a message to the SNS topic. This helps keep both patients and administrators informed about system activities.

troven.in/students/labs/68 x Subscriptions | Simple No x EC2 Instance Connect | us x SkillWallet x troven.in/students/labs/68 x MedTrack • Patient Dashb x + -

us-east-1.console.aws.amazon.com/sns/v3/home?region=us-east-1#/subscriptions

Google Chrome isn't your default browser [Set as default](#)

aws Search [Alt+S] United States (N. Virginia) Account ID: 6502-5170-3382 rsoaccount-new/68034028d24717cbb9270727

Amazon SNS > Subscriptions

Amazon SNS

- Dashboard
- Topics
- Subscriptions

▼ **Mobile**

- Push notifications
- Text messaging (SMS)

New Feature
Amazon SNS now supports High Throughput FIFO topics. [Learn more](#)

Subscriptions (1) [Edit](#) [Delete](#) [Request confirmation](#) [Confirm subscription](#) [Create subscription](#)

Search

ID	Endpoint	Status	Protocol	Topic
6f59f1de-da7f-4fc4-bf4c-c...	adivya140205@gmail.com	Confirmed	EMAIL	MEDTRACK

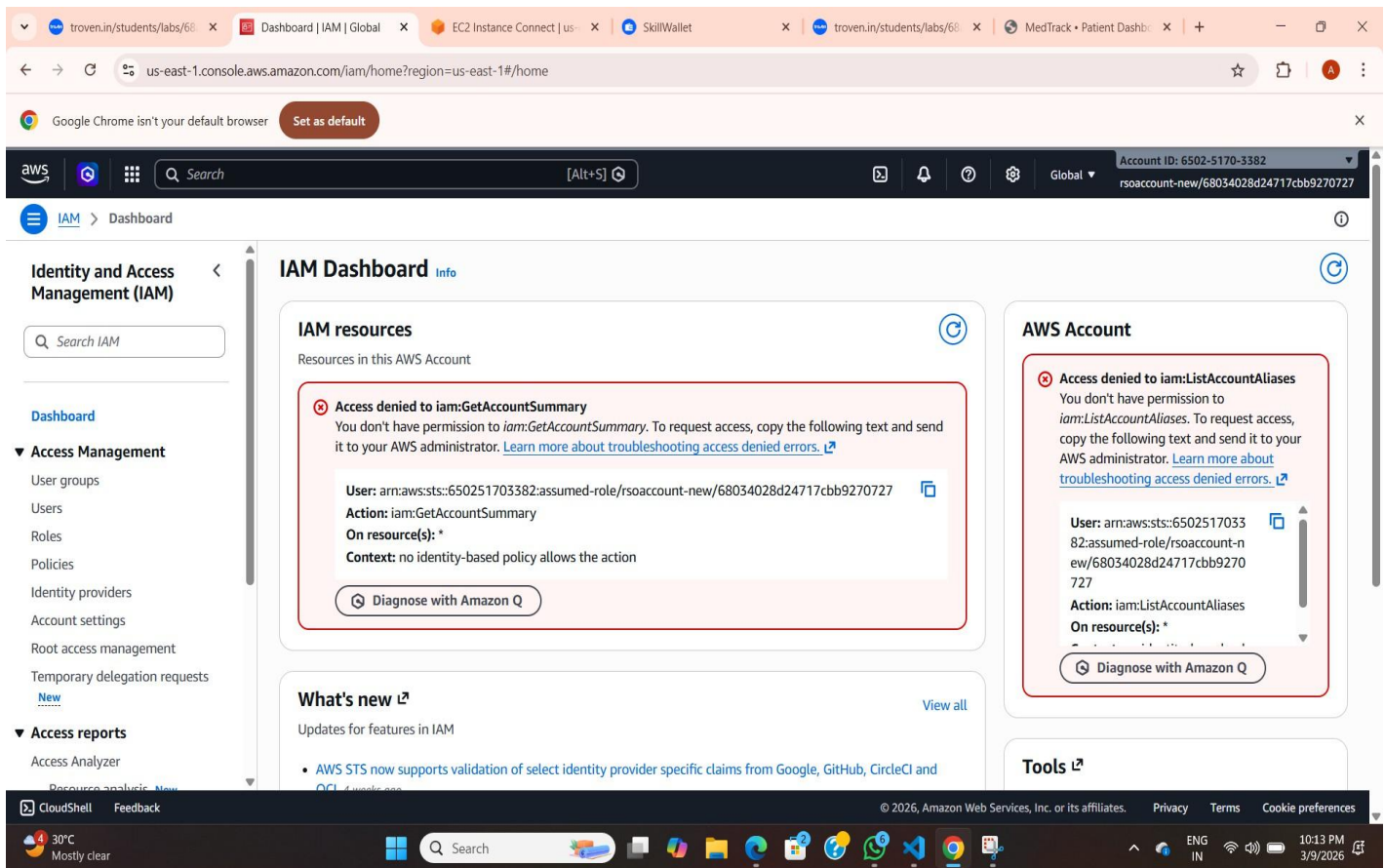
CloudShell Feedback © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

30°C Mostly clear Search ENG IN 10:14 PM 3/9/2026

Epic 5: IAM Role Setup

Activity 5.1 – Create IAM Role

AWS Identity and Access Management (IAM) is used to control access to AWS resources. An IAM role is created in the AWS Management Console to allow the EC2 instance to interact with other AWS services such as DynamoDB and SNS. This role ensures secure access and proper permissions for the application to function correctly.



The screenshot shows the AWS IAM Dashboard in a web browser. The browser tabs include 'troven.in/students/labs/60', 'Dashboard | IAM | Global', 'EC2 Instance Connect | us-', 'SkillWallet', 'troven.in/students/labs/60', and 'MedTrack - Patient Dashb'. The address bar shows 'us-east-1.console.aws.amazon.com/iam/home?region=us-east-1#/home'. The AWS account ID is 6502-5170-3382.

The IAM Dashboard is displayed with the following sections:

- Identity and Access Management (IAM)** sidebar with a search bar and navigation links: Dashboard, Access Management (User groups, Users, Roles, Policies, Identity providers, Account settings, Root access management, Temporary delegation requests), and Access reports (Access Analyzer, Resource analysis).
- IAM Dashboard Info** header.
- IAM resources** section:
 - Access denied to iam:GetAccountSummary**: You don't have permission to `iam:GetAccountSummary`. To request access, copy the following text and send it to your AWS administrator. [Learn more about troubleshooting access denied errors.](#)

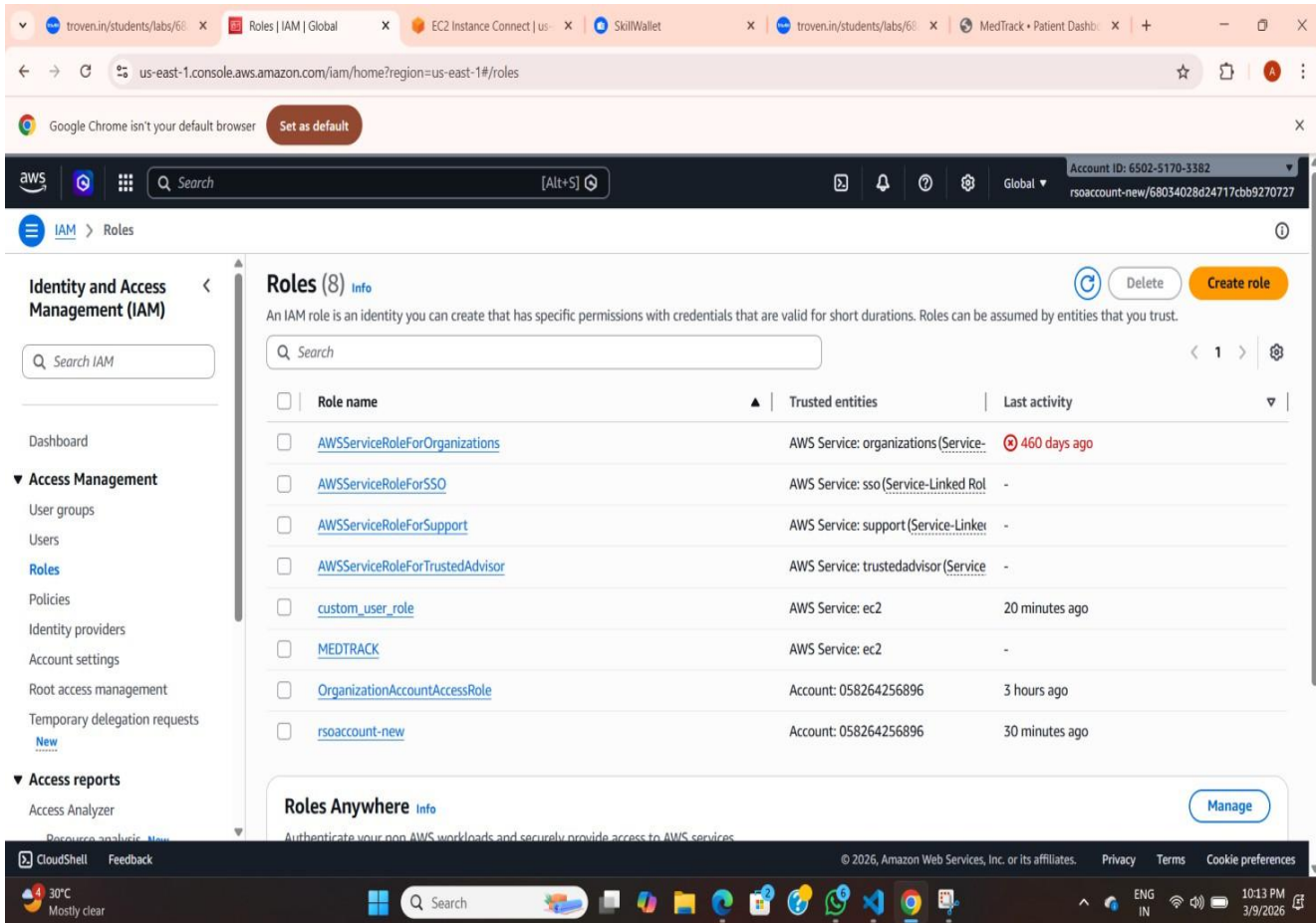
```
User: arn:aws:sts::650251703382:assumed-role/rsoaccount-new/68034028d24717cbb9270727
Action: iam:GetAccountSummary
On resource(s): *
Context: no identity-based policy allows the action
```
 - What's new**: Updates for features in IAM. Includes a link to [AWS STS now supports validation of select identity provider specific claims from Google, GitHub, CircleCI and](#).
- AWS Account** section:
 - Access denied to iam:ListAccountAliases**: You don't have permission to `iam:ListAccountAliases`. To request access, copy the following text and send it to your AWS administrator. [Learn more about troubleshooting access denied errors.](#)

```
User: arn:aws:sts::650251703382:assumed-role/rsoaccount-new/68034028d24717cbb9270727
Action: iam:ListAccountAliases
On resource(s): *
```
- Tools** section.

The bottom of the screen shows a Windows taskbar with the date and time 10:13 PM 3/9/2026, and a system tray with weather (30°C Mostly clear) and network status.

Activity 5.2 – Attach Policies to IAM Role

After creating the IAM role, required policies are attached to grant necessary permissions. Policies such as Amazon DynamoDB Full Access and Amazon SNS Full Access are added so that the application can read and write data in DynamoDB and send notifications using SNS.



The screenshot shows the AWS IAM console interface. The left sidebar contains navigation links for Identity and Access Management (IAM), Access Management, and Access reports. The main content area displays a list of roles under the heading "Roles (8)".

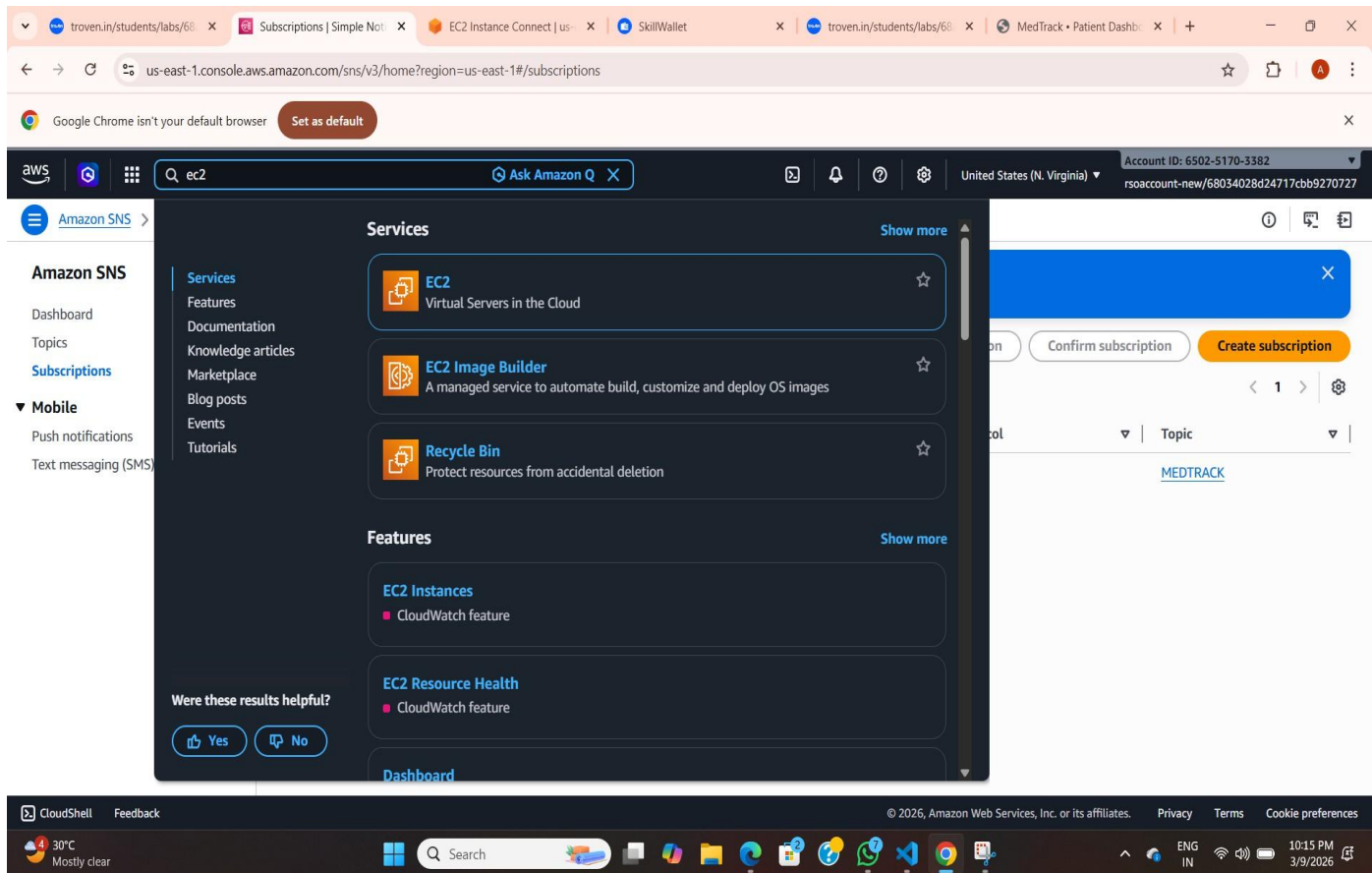
Role name	Trusted entities	Last activity
AWSServiceRoleForOrganizations	AWS Service: organizations (Service-Linked Role)	460 days ago
AWSServiceRoleForSSO	AWS Service: sso (Service-Linked Role)	-
AWSServiceRoleForSupport	AWS Service: support (Service-Linked Role)	-
AWSServiceRoleForTrustedAdvisor	AWS Service: trustedadvisor (Service-Linked Role)	-
custom_user_role	AWS Service: ec2	20 minutes ago
MEDTRACK	AWS Service: ec2	-
OrganizationAccountAccessRole	Account: 058264256896	3 hours ago
rsoaccount-new	Account: 058264256896	30 minutes ago

At the bottom of the console, there is a section for "Roles Anywhere" with a "Manage" button. The footer of the console shows the copyright notice: © 2026, Amazon Web Services, Inc. or its affiliates. The Windows taskbar at the bottom indicates the time is 10:13 PM on 3/9/2026.

Epic6: EC2 Instance Setup

Activity 6.1 – Launch EC2 Instance

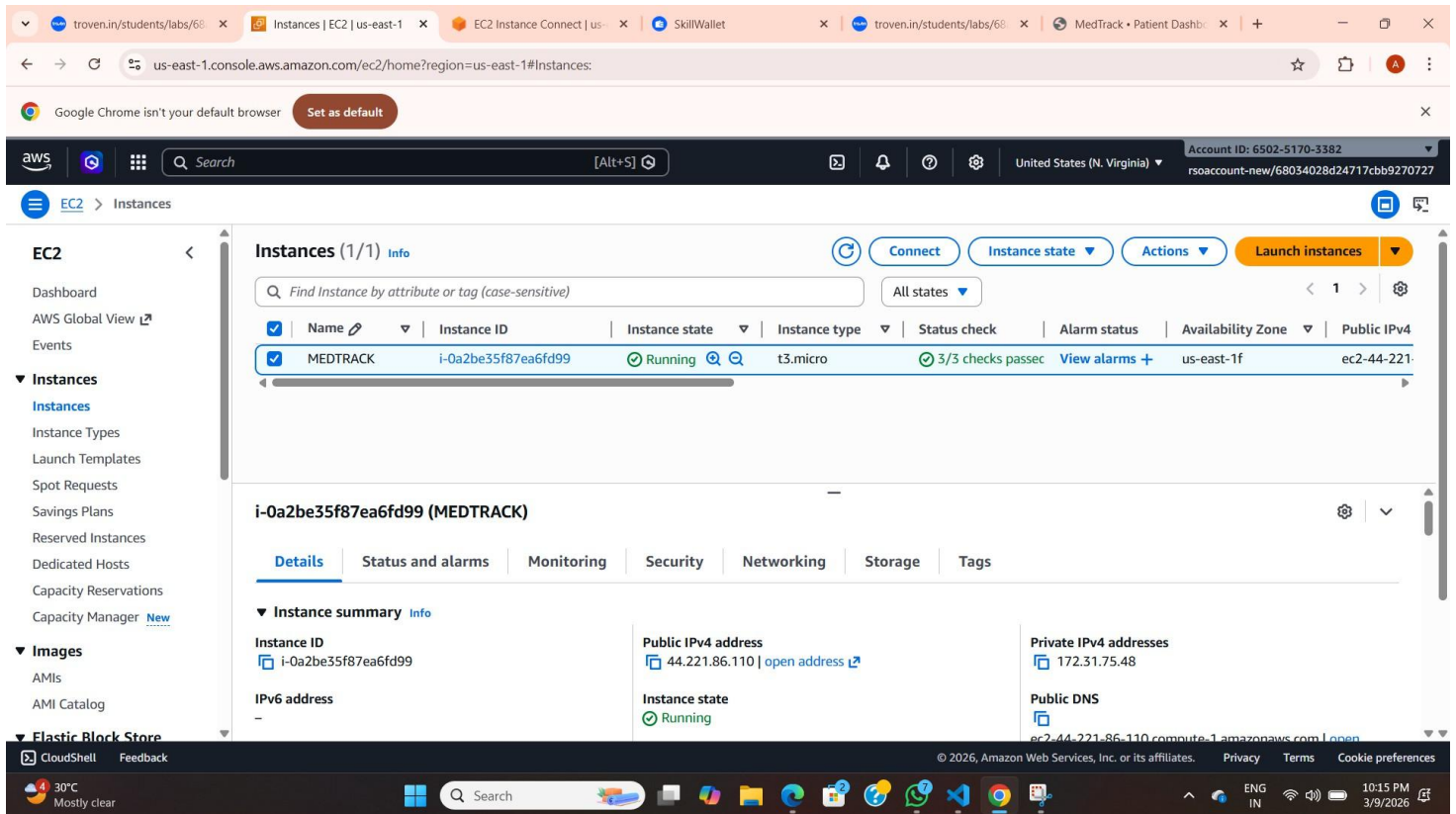
Amazon EC2 is used to host the MedTrack application on the cloud. In the AWS Management Console, the EC2 service is searched and selected. A new EC2 instance is launched by selecting an appropriate Amazon Machine Image (AMI) and instance type. This instance will run the Flask application and handle user requests.



The screenshot displays the AWS Management Console interface. The browser address bar shows the URL: `us-east-1.console.aws.amazon.com/sns/v3/home?region=us-east-1#/subscriptions`. The console header includes the AWS logo, a search bar with the text 'ec2', and the account ID '6502-5170-3382'. The left sidebar shows the 'Amazon SNS' navigation menu. The main content area displays search results for 'ec2', including the 'EC2' service (Virtual Servers in the Cloud), 'EC2 Image Builder', and 'Recycle Bin'. Below the services, there are sections for 'Features' and 'EC2 Instances' (CloudWatch feature). At the bottom, there is a feedback prompt: 'Were these results helpful?' with 'Yes' and 'No' buttons. The bottom of the screen shows a Windows taskbar with the date and time '10:15 PM 3/9/2026'.

Activity 6.2 – Configure Security Groups and Instance Settings

After launching the EC2 instance, security groups are configured to allow necessary network access. Ports such as SSH (22), HTTP (80), HTTPS (443), and custom application port (5000) are enabled so that the application can be accessed securely and remotely.



The screenshot displays the AWS Management Console for the 'us-east-1' region. The 'Instances' page shows a single instance named 'MEDTRACK' with ID 'i-0a2be35f87ea6fd99'. The instance is in a 'Running' state and is of type 't3.micro'. The instance details section shows the Public IPv4 address as 44.221.86.110 and the Instance state as 'Running'.

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4
MEDTRACK	i-0a2be35f87ea6fd99	Running	t3.micro	3/3 checks passed	View alarms +	us-east-1f	ec2-44-221-86-110

i-0a2be35f87ea6fd99 (MEDTRACK)

Details | Status and alarms | Monitoring | Security | Networking | Storage | Tags

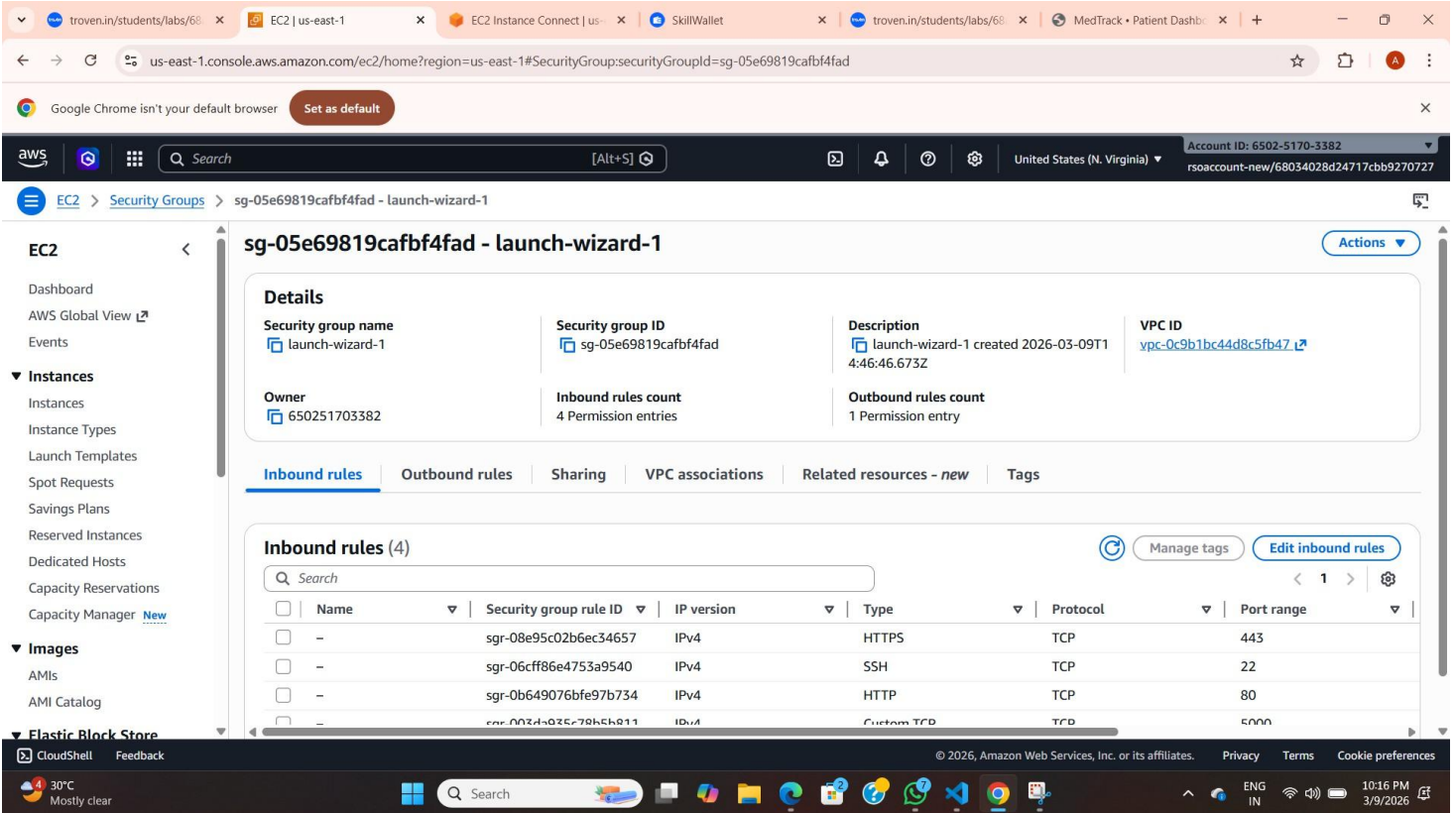
Instance summary

Instance ID i-0a2be35f87ea6fd99	Public IPv4 address 44.221.86.110 open address	Private IPv4 addresses 172.31.75.48
IPv6 address -	Instance state Running	Public DNS ec2-44-221-86-110.compute-1.amazonaws.com open

Epic 7: Security Group Configuration

Activity 7.1 – View Security Group Details

After launching the EC2 instance, the security group attached to the instance is checked. Security groups act as virtual firewalls that control inbound and outbound traffic for the EC2 instance. The security group settings are reviewed to ensure the instance can communicate with required services.



us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#SecurityGroup:securityGroupId=sg-05e69819cafbf4fad

Google Chrome isn't your default browser [Set as default](#)

aws [Search](#) [Alt+S] [United States \(N. Virginia\)](#) Account ID: 6502-5170-3382 rsoaccount-new/68034028d24717cbb9270727

EC2 > Security Groups > sg-05e69819cafbf4fad - launch-wizard-1

sg-05e69819cafbf4fad - launch-wizard-1

Details

Security group name launch-wizard-1	Security group ID sg-05e69819cafbf4fad	Description launch-wizard-1 created 2026-03-09T14:46:46.673Z	VPC ID vpc-0c9b1bc44d8c5fb47
Owner 650251703382	Inbound rules count 4 Permission entries	Outbound rules count 1 Permission entry	

[Inbound rules](#) | [Outbound rules](#) | [Sharing](#) | [VPC associations](#) | [Related resources - new](#) | [Tags](#)

Inbound rules (4)

[Manage tags](#) [Edit inbound rules](#)

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range
☐	-	sgr-08e95c02b6ec34657	IPv4	HTTPS	TCP	443
☐	-	sgr-06cff86e4753a9540	IPv4	SSH	TCP	22
☐	-	sgr-0b649076bfe97b734	IPv4	HTTP	TCP	80
☐	-	sgr-003d4025e70b5b011	IPv4	Custom TCP	TCP	5000

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

CloudShell Feedback

30°C Mostly clear

10:16 PM 3/9/2026

Activity 7.2 – Configure Inbound Rules

Inbound rules are configured in the security group to allow necessary ports for accessing the application. Ports such as SSH (22) for remote access, HTTP (80) and HTTPS (443) for web access, and custom application port (5000) for the Flask application are enabled

troven.in/students/labs/6i x ModifyInboundSecurityGro x EC2 Instance Connect | us x SkillWallet x troven.in/students/labs/6i x MedTrack - Patient Dashb x +

us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#ModifyInboundSecurityGroupRules:securityGroupId=sg-05e69819cafbf4fad

Google Chrome isn't your default browser [Set as default](#)

Account ID: 6502-5170-3382
rsaccount-new/68034028d24717cbb9270727

EC2 > Security Groups > sg-05e69819cafbf4fad - launch-wizard-1 > Edit inbound rules

Edit inbound rules [Info](#)

Inbound rules control the incoming traffic that's allowed to reach the instance.

Security group rule ID	Type Info	Protocol Info	Port range Info	Source Info	Description - optional Info	
sg-08e95c02b6ec34657	HTTPS	TCP	443	Custom	Q	Delete
sg-06cff86e4753a9540	SSH	TCP	22	Custom	Q	Delete
sg-0b649076bfe97b734	HTTP	TCP	80	Custom	Q	Delete
sg-003da935c78b5b811	Custom TCP	TCP	5000	Custom	Q	Delete

[Add rule](#)

Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

CloudShell Feedback © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

30°C Mostly clear 10:16 PM 3/9/2026

Epic 8: Deployment on EC2

Activity 7.1: Install Software on the EC2 Instance

Update and install Python + pip

```
sudo apt update
```

```
sudo apt install python3-pip -y
```

```
sudo apt install git -y
```

```
sudo apt install python3-venv -y
```

```
python3 -m venv venv
```

```
source venv/bin/activate
```

Activity 7.2: Clone Your Flask Project from GitHub

Clone your project repository from GitHub into the EC2 instance using Git.

Run: `git clone https://github.com/coder-PRAKASH/medtrack-aws-prakash.git`

Note: change your-github-username and your-repository-name with your credentials

here: `'https://github.com/AlekhyaPenubakula/Travel-Booking-website_Ec2_Dynamodb_SNS.git'`

- This will download your project to the EC2 instance.

To navigate to the project directory, run the following command:

```
cd <your-folder>
```

Once inside the project directory, configure and run the Flask application by executing the following command with elevated privileges:

Run the Flask Application

```
export SNS_TOPIC_ARN=arn:aws:sns:ap-south-1:557690616836:BookingConfirmation
```

```
git pull origin main ( Only if you made changes in git and want to reflect them in the EC2 terminal.)
```

Install requirements

```
pip install flask boto3
```

Launch the Flask app:

```
sudo -E venv/bin/python3 app.py
```

Verify the Flask app is running:

<http://your-ec2-public-ip>

- Run the Flask app on the EC2 instance

```
Last login: Mon Apr 14 16:27:44 2025 from 13.233.177.5
ubuntu@ip-172-31-12-110:~$ cd Travel-Booking-website_Ec2_Dynamodb_SNS
ubuntu@ip-172-31-12-110:~/Travel-Booking-website_Ec2_Dynamodb_SNS$ cd travel_booking_system
ubuntu@ip-172-31-12-110:~/Travel-Booking-website_Ec2_Dynamodb_SNS/travel_booking_system$ export SNS_TOPIC_ARN=arn:aws:sns:ap-south-1:557690616836:BookingConfirmation
ubuntu@ip-172-31-12-110:~/Travel-Booking-website_Ec2_Dynamodb_SNS/travel_booking_system$ git pull origin main
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 5 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (5/5), 1.68 KiB | 429.00 KiB/s, done.
From https://github.com/Alekhyapenubakula/Travel-Booking-website_Ec2_Dynamodb_SNS
 * branch          main       -> FETCH_HEAD
   2289f47..397e3a1 main     -> origin/main
Updating 2289f47..397e3a1
Fast-forward
 travel_booking_system/templates/hotels.html | 107 ++++++
 1 file changed, 63 insertions(+), 44 deletions(-)
ubuntu@ip-172-31-12-110:~/Travel-Booking-website_Ec2_Dynamodb_SNS/travel_booking_system$ sudo -E venv/bin/python3 app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:80
 * Running on http://172.31.12.110:80
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 661-186-119
```

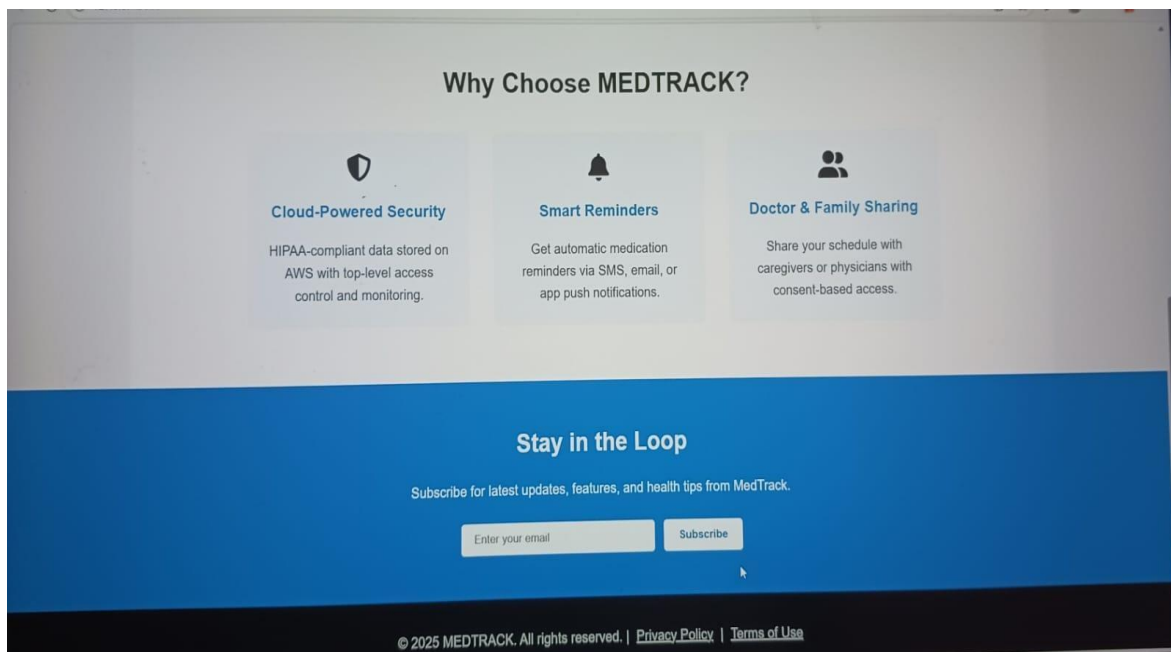
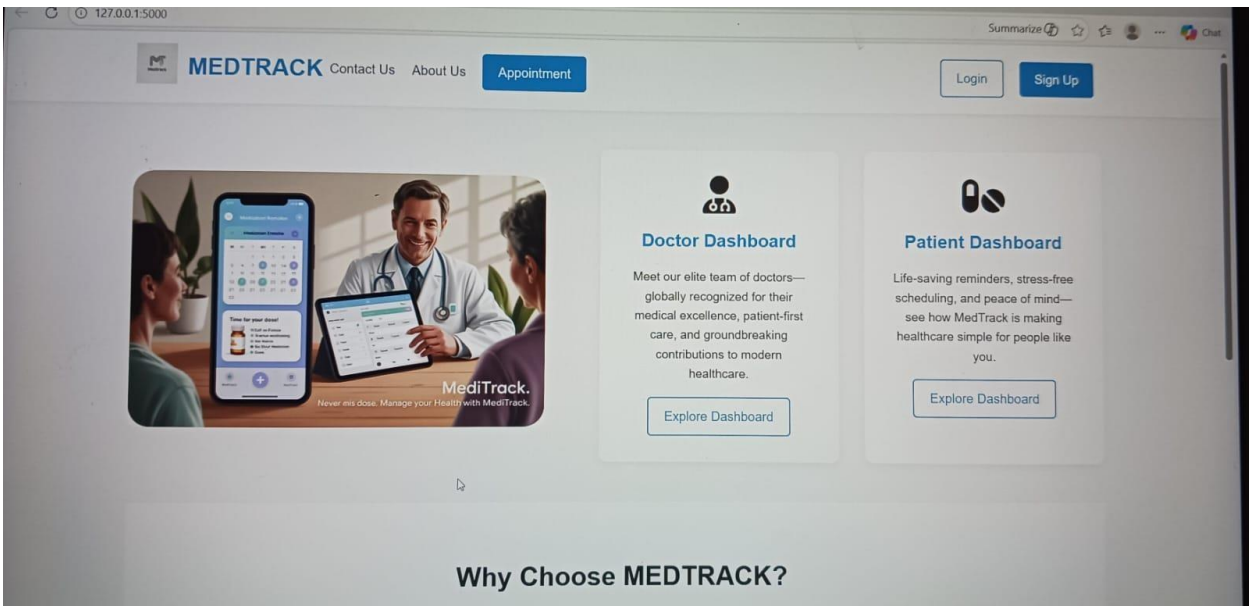
Access the website through:

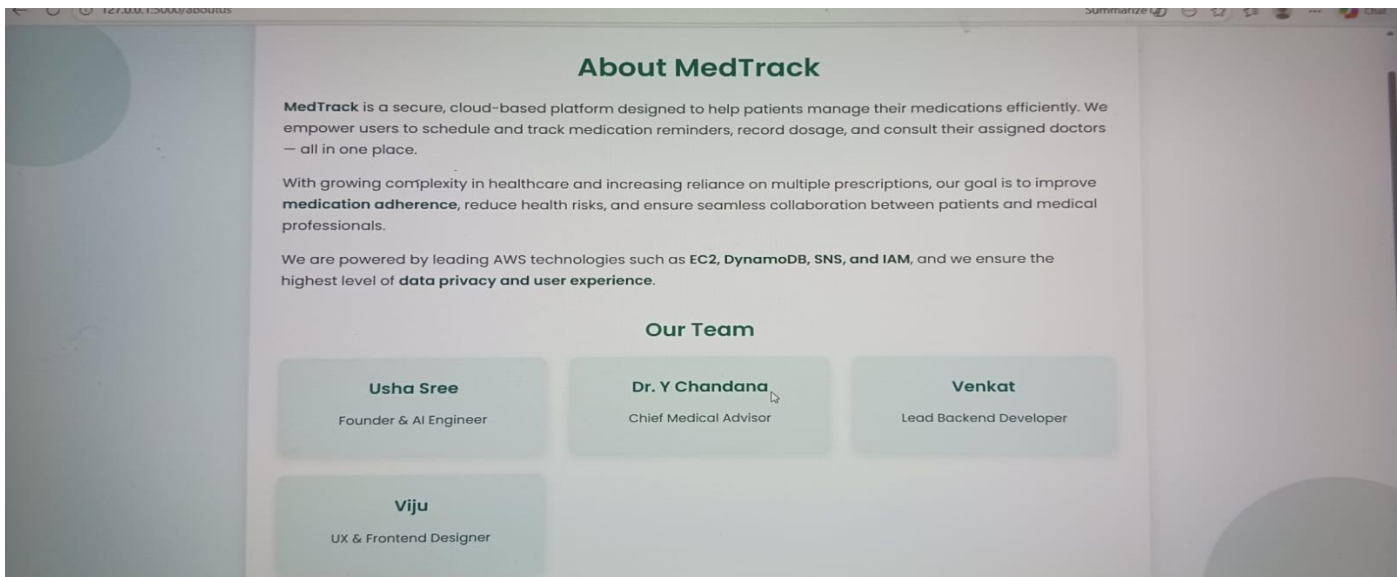
PublicIPS: <http://44.221.86.110:5000>

Milestone 8: Testing and Deployment

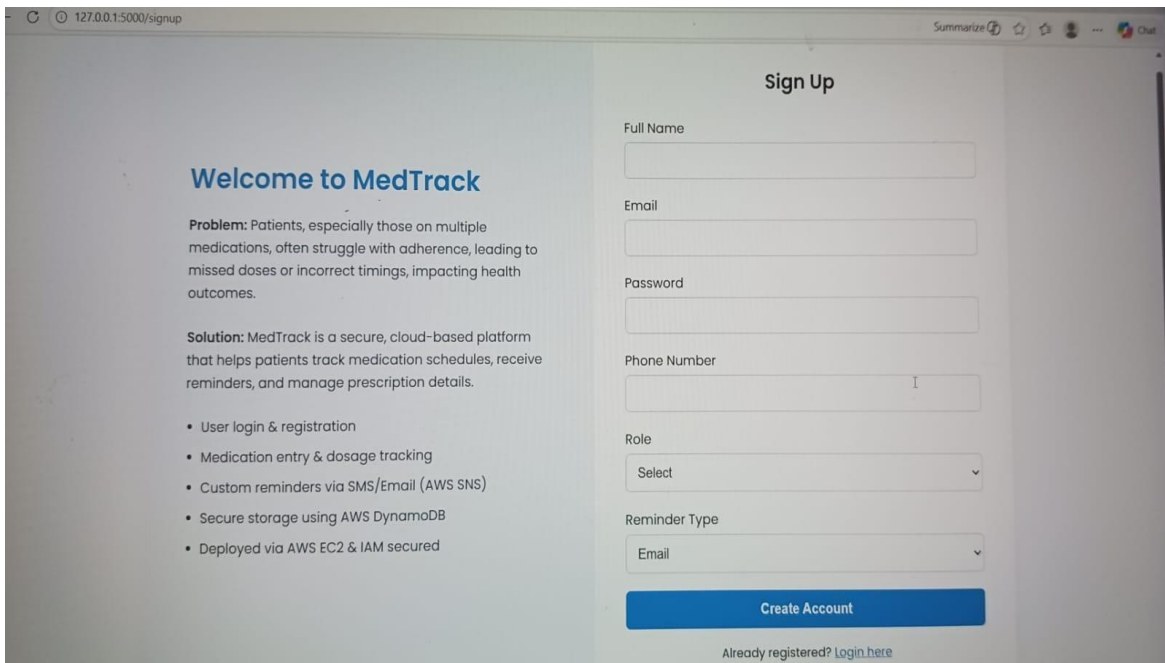
- **Activity 8.1: Conduct functional testing to verify user registration, login, book requests, and notifications.**

Home Page:





Register Page:



Welcome to MedTrack

Problem: Patients, especially those on multiple medications, often struggle with adherence, leading to missed doses or incorrect timings, impacting health outcomes.

Solution: MedTrack is a secure, cloud-based platform that helps patients track medication schedules, receive reminders, and manage prescription details.

- User login & registration
- Medication entry & dosage tracking
- Custom reminders via SMS/Email (AWS SNS)
- Secure storage using AWS DynamoDB
- Deployed via AWS EC2 & IAM secured

Sign Up

Full Name

Email

Password

Phone Number

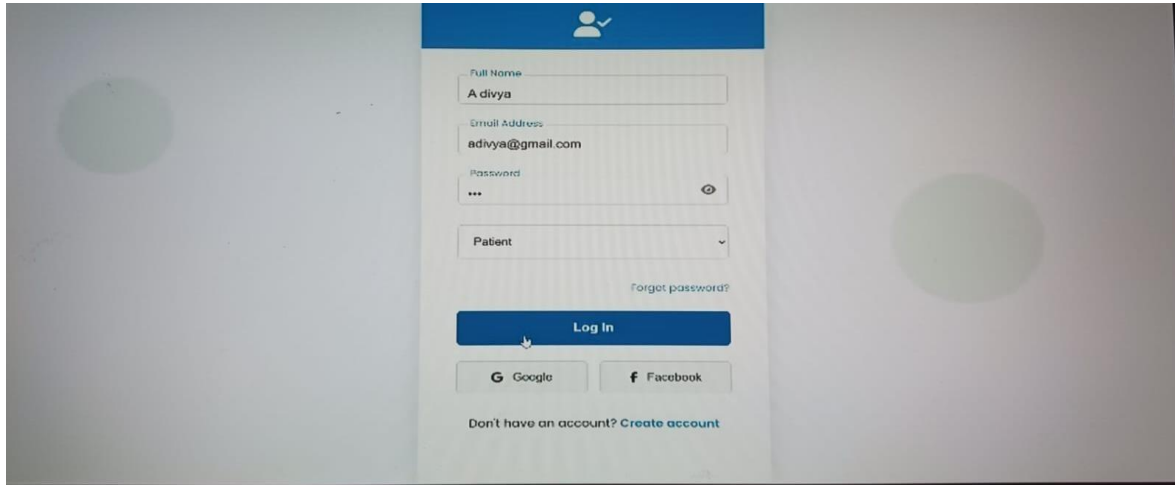
Role

Reminder Type

Create Account

Already registered? [Login here](#)

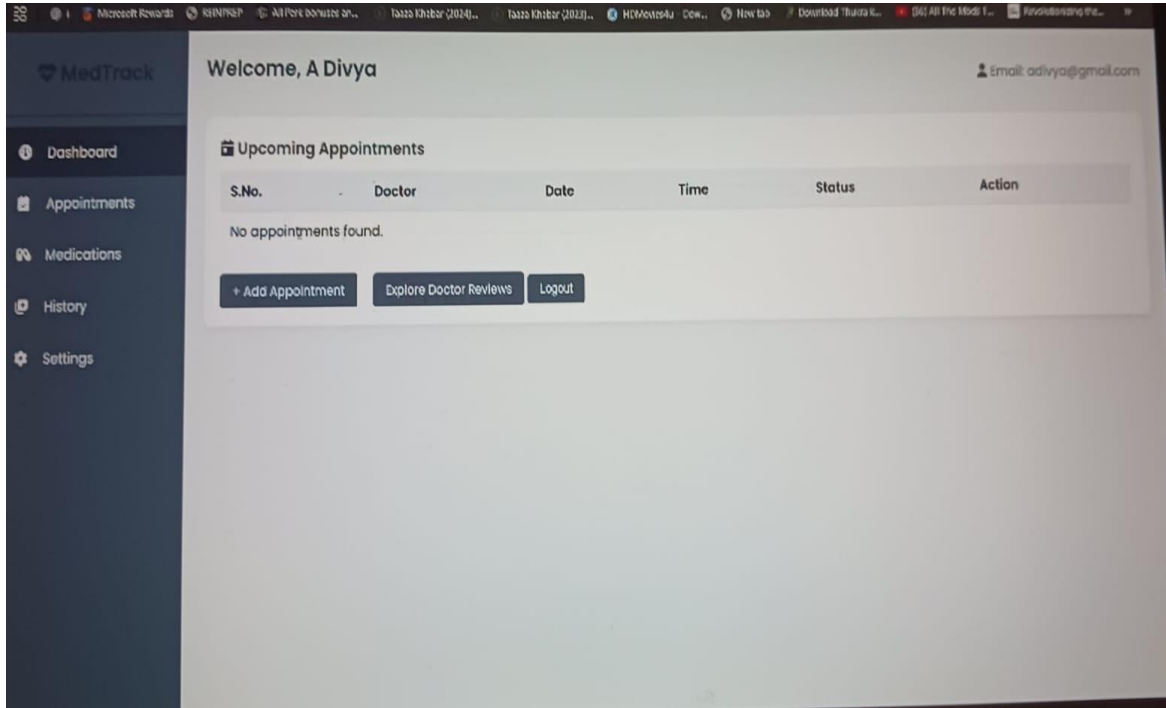
Register Page:



The screenshot shows a registration form on a mobile device. At the top, there is a blue header with a white user icon and a checkmark. The form fields are as follows:

- Full Name:** A divya
- Email Address:** aditya@gmail.com
- Password:** masked with three dots, with an eye icon to toggle visibility.
- Patient:** a dropdown menu.
- Forgot password?** a link below the password field.
- Log In:** a blue button with a white arrow pointing to it.
- Google:** a button with the Google logo.
- Facebook:** a button with the Facebook logo.
- Don't have an account? Create account:** a link at the bottom.

Dashboard page:



The screenshot shows the MedTrack dashboard for a user named A Divya. The interface includes a dark sidebar with navigation links: Dashboard, Appointments, Medications, History, and Settings. The main content area has a welcome message and a section for 'Upcoming Appointments'. This section contains a table with columns for S.No., Doctor, Date, Time, Status, and Action. Below the table, it states 'No appointments found.' and provides three buttons: '+ Add Appointment', 'Explore Doctor Reviews', and 'Logout'. The user's email, 'adivya@gmail.com', is displayed in the top right corner.

MedTrack

Welcome, A Divya

Email: adivya@gmail.com

Upcoming Appointments

S.No.	Doctor	Date	Time	Status	Action
No appointments found.					

+ Add Appointment Explore Doctor Reviews Logout

Full Name
divya

Gender
Female

Phone Number
1234567890

Age
22

Department / Specialization
-- Select Department --
-- Select Department --
Gynecology
Orthopedic
Cardiology
Neurology
Dermatology
-- Select Doctor --

Preferred Date

Department / Specialization
Neurology

Describe Your Problem
Having a serious headache

Select Available Doctor
Ravi Teja

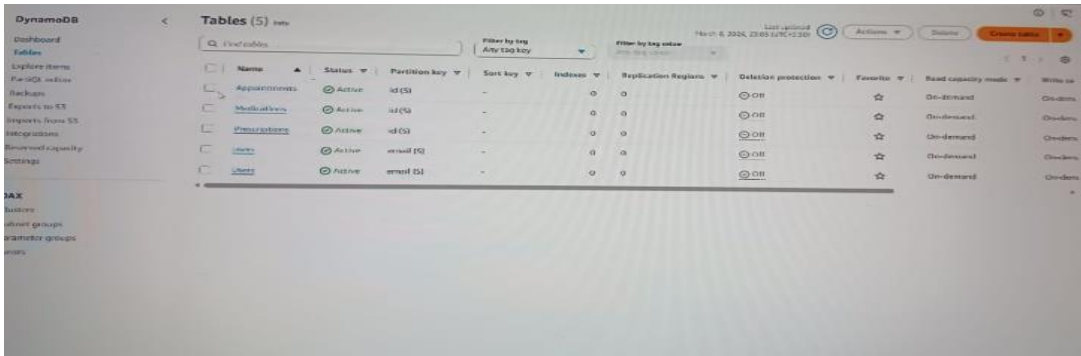
Preferred Date
10-03-2026

Preferred Time
10:30 AM

Book Appointment

Dynamodb Database updations :

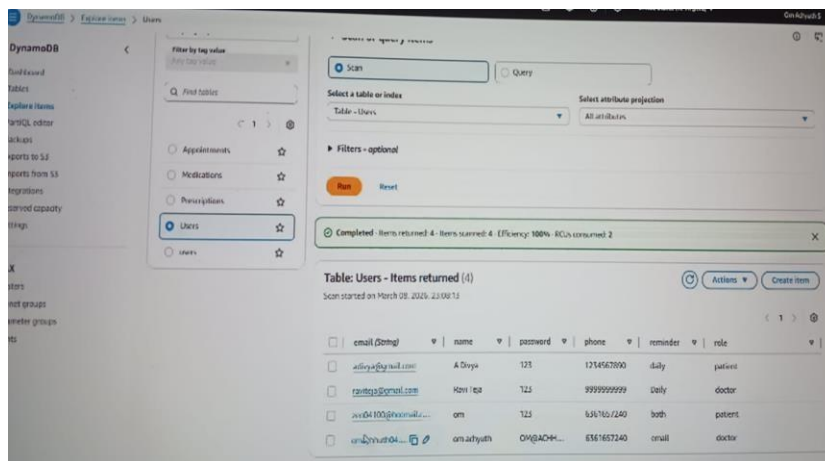
1. Patient-users table :



Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Favorite	Read capacity mode	Write capacity mode
Appointments	Active	id (S)	-	0	0	Off	☆	On-demand	On-demand
Medications	Active	id (S)	-	0	0	Off	☆	On-demand	On-demand
Prescriptions	Active	id (S)	-	0	0	Off	☆	On-demand	On-demand
Users	Active	email (S)	-	0	0	Off	☆	On-demand	On-demand
Users	Active	email (S)	-	0	0	Off	☆	On-demand	On-demand

Description: Store and manage user credentials, login counts, and names in the **Users** table on DynamoDB for authentication and tracking purposes.

2. bookings table :



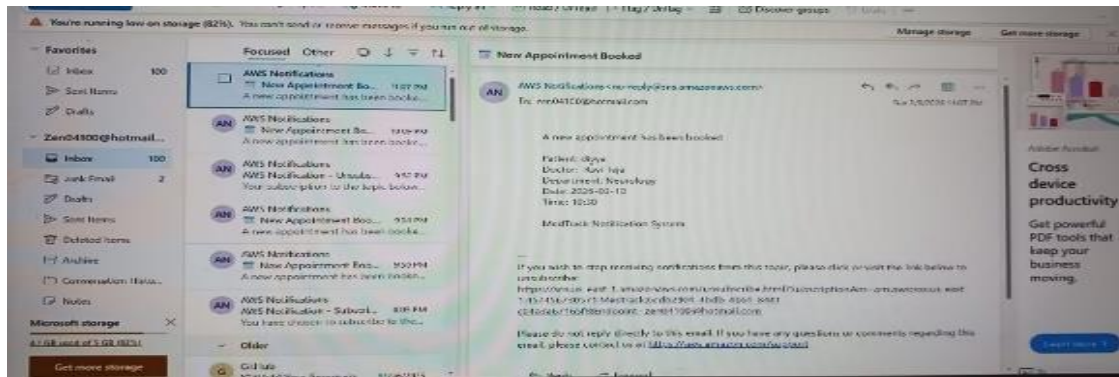
Completed: Items returned: 4 - Items scanned: 4 - Efficiency: 100% - RCUs consumed: 2

Table: Users - Items returned (4)
Scan started on March 08, 2024, 23:00:13

email (string)	name	password	phone	reminder	role
aditya@gmail.com	A Ditya	123	1234567890	daily	patient
rajesh@gmail.com	Rajesh	123	9999999999	daily	doctor
rajesh100@gmail.com	rajesh	123	6361627240	both	patient
aditya@gmail.com	aditya	OP@ACH...	6361627240	oncall	doctor

Description: Use the Bookings table in DynamoDB to store user travel bookings including date, destination, transport details, and payment information for each transaction.

3. Mail to the User:



Conclusion:

The MEDTRACK Healthcare Management System has been successfully designed and deployed using a cloud-based architecture on Amazon Web Services (AWS). The system integrates various AWS services such as EC2 for hosting the application, DynamoDB for efficient data storage, SNS for sending email notifications, and IAM for secure access management. By leveraging these cloud services, the platform ensures scalability, reliability, and secure handling of healthcare data.

The application provides an easy-to-use interface where patients can register, log in, and book appointments with doctors. Doctors can view appointment requests, access patient information, and update diagnosis reports. The system improves communication between patients and healthcare providers by providing a centralized platform for managing appointments and medical records.

Using Flask as the backend framework allows the application to efficiently process user requests and manage interactions between the web interface and AWS cloud services. DynamoDB enables fast and reliable storage of user and appointment data, while SNS ensures timely notifications to keep users informed about important system updates.

Overall, the MEDTRACK system demonstrates how cloud computing technologies can be effectively used to develop modern healthcare management solutions. The project highlights the benefits of using scalable cloud infrastructure to build secure, reliable, and user-friendly applications that enhance the efficiency of healthcare service.